

Name der Hochschule

Studiengang

Modulname

SRD Arena

Entwicklung einer Engine für taktische Kämpfe nach dem
SRD 5.2.1

Autor:	Vorname Nachname
Matrikelnummer:	00000000
Lehrperson:	Name der Lehrperson
Abgabedatum:	30. August 2026

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Zielsetzung	2
1.2.1	Verwendete Pakete	2
1.2.2	Abgrenzung des MVP	2
1.2.3	Nächste Entscheidungen zur Implementierungsstrategie	5
1.2.4	Meilensteine	5
1.2.5	Erwartete Schwierigkeiten	6
2	Theoretische Grundlagen und Werkzeuge	7
2.1	Technologien	7
2.2	Bibliotheken und Pakete	7
2.3	Mensch und KI	8
3	Konzept und Implementierung	9
3.1	Programmstruktur	9
3.2	Kernlogik	9
3.3	Datenverarbeitung	9
3.4	Architektur- und Designentscheidungen	9
3.4.1	Content-Schicht	10
3.5	KI-Konsultation bei Designentscheidungen	10
3.6	Code-Generierung	11
3.6.1	KI-generierte Bestandteile	11
3.6.2	Selbst programmierte Bestandteile	12
3.7	Prompt-Engineering und Iteration	12
4	Ergebnisse	13
4.1	Funktionsnachweis	13
4.2	Beispieldurchlauf	13
5	Diskussion und Fazit	14
5.1	Zielerreichung	14
5.2	Herausforderungen und kritische Reflexion	14

5.3	Abweichungen vom ursprünglichen Implementierungsplan	14
5.4	Kritische Würdigung	14
5.5	Ausblick	14
6	Literatur- und Quellenverzeichnis	15
7	Anhang	16
7.1	Bedienungsanleitung	16
7.2	Ausgewählte Code-Snippets	16

1 Einleitung

Dungeons & Dragons (D&D) ist ein beliebtes Pen-and-Paper-Rollenspiel, in dem taktische, rundenbasierte Kämpfe (Encounter) einen großen Teil des Spiels ausmachen. Üblicherweise kämpft ein Team von Spielern gegen ein Team von NPCs (Non-Player Characters), die vom Spielleiter gesteuert werden.

Das System Reference Document (SRD) 5.2.1 ist eine unter der Creative-Commons-Lizenz CC BY 4.0 verfügbare Fassung der Spielregeln. Sie enthält den größten Teil der Grundmechaniken des Spiels, jedoch nicht sämtliche Inhalte der proprietären Regelwerke. So kann ein Spieler beispielsweise eine Klasse und Subklasse wählen; im SRD steht aber pro Klasse nur eine Subklasse zur Verfügung, während das proprietär lizenzierte Spielerhandbuch weitere Subklassen enthält.

SRD Arena setzt zunächst einen Teil des Kampfsystems des SRD 5.2.1 als rundenbasiertes 2D-Spiel um. Mehrere vordefinierte Kampfszenarien demonstrieren die implementierten Funktionen. Mithilfe von Vorlagen können Nutzer weitere Szenarien in strukturierten Dateien definieren und anschließend spielen.

1.1 Motivation

Die meisten Spielleiter entwerfen Kampfszenarien bereits vor dem eigentlichen Spieltermin. Ihr Ziel ist es, den Spielern neben einem interessanten Narrativ auch eine taktische Herausforderung zu bieten, ohne den Kampf unangemessen schwer zu gestalten.

Im Rahmen der umgesetzten Regeln können Spielleiter mit SRD Arena ihre Kampfszenarien vor dem Spiel ausprobieren. Viele Berechnungen und Würfelwürfe, die sonst manuell erfolgen müssten, werden dabei automatisiert. Mittelfristig, jedoch nach Abgabe des Projekts, soll der gesamte Umfang der SRD-Regeln abgedeckt werden.

Langfristig könnte das Projekt um Bots erweitert werden, die die Kampfsentscheidungen aller Teilnehmer automatisieren. Kämpfe könnten dann vielfach simuliert und die entstehenden Daten in robuste Kennzahlen zur Ausgewogenheit eines Szenarios überführt werden. Diese Funktionalität stellt die weiterführende Motivation des Projekts dar,

kann aufgrund der begrenzten Bearbeitungszeit jedoch erst nach der Abgabe umgesetzt werden.

1.2 Zielsetzung

SRD Arena ist eine Engine für taktische Kämpfe auf einem 2D-Spielfeld nach dem SRD-5.2.1-Regelwerk. Charaktere werden wahlweise durch Nutzereingaben oder durch einfache Bots gesteuert. Szenarien werden in strukturierten Textdateien definiert; dafür stehen eine Vorlage und mehrere Demonstrationsszenarien zur Verfügung.

1.2.1 Verwendete Pakete

Kernabhängigkeiten

- `pydantic`: Validierung der JSON-Daten
- `PySide6`: grafische Benutzeroberfläche
- `scipy`: geometrische Berechnungen

Entwicklungsabhängigkeiten

- `hypothesis`: Property-Based Testing
- `mypy`: statische Typprüfung
- `pytest`: Unit-Tests
- `pytest-cov`: Messung der Testabdeckung
- `ruff`: Linter und Formatter

1.2.2 Abgrenzung des MVP

Wesentlich ist eine stabile Encounter-Engine für vorhandene Spieldaten. Der MVP umfasst ein klar abgegrenztes Subset der SRD-Kampfmechaniken.

Kampfablauf

- Initiative
- Zugstruktur
- Action, Bonus Action und Movement
- Opportunity Attack als erste Reaktionsmechanik

Bewegung

- rasterbasierte Bewegung

Kampfmechaniken

- Nahkampf- und Fernkampfangriffe
- Zauber, deren zugrunde liegende Mechaniken unterstützt werden
- Attack Rolls und Saving Throws
- Armor Class und Critical Hits
- Waffen mit unterschiedlichen Schadenswürfeln

Effekte

- Radius- und Kegel-Flächeneffekte
- Heilung
- einfache Zustände, insbesondere *Blinded* und *Prone*

Nicht Teil des MVP

- vollständige SRD-Abdeckung
- vollständige Zauberliste
- vollständige Klassenprogression
- vollständige Monster-KI

Erfolgskriterien

- Szenarien und Spieldaten können aus JSON geladen werden.
- Ungültige Eingaben werden durch Schema-Validierung erkannt.
- Ein vollständiger Encounter lässt sich ausführen; anschließend wird das Textabenteuer fortgesetzt.
- Derselbe Seed erzeugt denselben Encounter-Verlauf.
- Ein Baseline-Agent spielt einen vollständigen Encounter mit ausschließlich legalen Aktionen.
- Aktionen, Würfe, Zustandsänderungen und Kampfende sind vollständig im Log enthalten.
- Alle unterstützten Mechaniken sind durch automatisierte Tests abgedeckt.

Optionale Erweiterungen

- weitere AoE-Typen und Flächeneffekte über mehrere Runden
- komplexere Zustände
- Summons, die neue Akteure erzeugen und in die Initiative aufnehmen
- weitere Reaktionsmechaniken
- Konzentration, Grapple und Shove
- Vision, Licht, Hindernisse und Line of Sight
- bessere Gegner-Agenten

- aufwendigere GUI-Animationen

1.2.3 Nächste Entscheidungen zur Implementierungsstrategie

- Feature-Abhängigkeitsgraph und Abhängigkeiten zwischen Mechaniken dokumentieren
- spätere Erweiterungen priorisieren
- JSON-Schemas finalisieren
- Formate für Spieldaten, Szenarien und Traces festlegen
- Validierungsregeln definieren
- Schnittstelle zwischen Kernlogik und GUI festlegen
- Regelentscheidungen und Zustandsverwaltung in der Kernlogik belassen
- GUI auf Darstellung und Eingabemöglichkeiten begrenzen

1.2.4 Meilensteine

- Kernlogik aufräumen
- JSON-Schemas finalisieren
- Schnittstelle für Aktionen und Zielauswahl fertigstellen
- Determinismus sicherstellen
- Testabdeckung für alle MVP-Mechaniken herstellen
- Baseline-Agent stabilisieren
- GUI anbinden
- optionale Erweiterungen umsetzen

1.2.5 Erwartete Schwierigkeiten

- Reaktionen verkomplizieren die Zugstruktur, da andere Teilnehmer während eines fremden Zuges kurzfristig handeln können.
- AoE-Effekte erzeugen zahlreiche Randfälle; im MVP werden nur ausgewählte AoE-Typen unterstützt.
- JSON-Daten können formal gültig sein, obwohl die benötigte Mechanik noch nicht implementiert ist. Schema-Validierung und Laufzeitfehler müssen deshalb getrennt behandelt werden.
- Viele Mechaniken bauen aufeinander auf. Fehlende Grundmechaniken können mehrere spätere Erweiterungen blockieren.
- GUI und Gegner-Agenten müssen mit wachsendem Funktionsumfang angepasst werden.

2 Theoretische Grundlagen und Werkzeuge

2.1 Technologien

TODO: Die eingesetzten Technologien einordnen und ihre Auswahl begründen.

2.2 Bibliotheken und Pakete

Vorgabe: Beschreiben, welche Pakete und Bibliotheken zum Einsatz kamen.

- Typprüfung: `mypy`
- Linting und Formatierung: `ruff`
- Tests: `pytest`
- Property-Tests: `hypothesis`
- Testabdeckung: `pytest-cov`
- Datenvalidierung: `pydantic`
- Paketverwaltung: `uv`
- Konfiguration: `pyproject.toml`
- Debugging und Replay: Logging und JSON-Traces
- Nicht-Python-Datenquelle: JSON-Daten der SRD-Regeln von `5e.tools` als Grundlage für den authored content

2.3 Mensch und KI

Vorgabe: Die Rolle der KI-Unterstützung im Projekt erläutern.

In diesem Projekt wurde ein großer Teil der Programmlogik mit KI-Unterstützung generiert. Viele Strukturideen wurden mithilfe der KI auf Lücken geprüft und iterativ angepasst. Außerdem wurden Regelabschnitte, die in JSON-Dateien zunächst nur als Prosa vorlagen, mithilfe der KI in strukturierte JSON-Definitionen übertragen.

3 Konzept und Implementierung

3.1 Programmstruktur

TODO: Die obersten Pakete und ihre Verantwortlichkeiten anhand einer Grafik oder eines kompakten Paketbaums erläutern.

3.2 Kernlogik

TODO: Zustandsmodell, Zugablauf und Ausführung von Aktionen beschreiben.

3.3 Datenverarbeitung

TODO: Laden, Validieren, Indizieren und Bauen der Inhaltsdaten beschreiben.

3.4 Architektur- und Designentscheidungen

Vorgabe: Begründen, warum die Software auf diese Weise strukturiert wurde. Dazu gehören beispielsweise die Wahl bestimmter Entwurfsmuster, objektorientierter Strukturen oder funktionaler Ansätze sowie verworfene Alternativen und deren Gründe.

Wesentliche Entscheidungen sind:

- authored content in JSON-Dateien
- Trennung in Content-, Domain-, Runtime- und GUI-Schicht

3.4.1 Content-Schicht

Listing 3.1: Vom Dateisystem zum Domain-Zauber

```
Dateisystem
  | load_spell_catalog()
  v
SpellCatalog mit validierten SpellSchema-Objekten
  | find()
  v
SpellSchema
  | build_spell()
  v
Domain-Objekt Spell
```

Listing 3.2: Validierung und Indizierung

```
JSON-Datei
  | parsen und validieren
  v
SpellSchema
  | speichern und indizieren
  v
SpellCatalog
```

3.5 KI-Konsultation bei Designentscheidungen

Vorgabe: Darstellen, ob die KI bereits bei Planung und Architektur konsultiert wurde, welche Vorschläge übernommen wurden und wo bewusst von Empfehlungen abgewichen wurde.

Die KI unterstützte insbesondere beim Erlernen und Anwenden einer geschichteten Architektur, beim Schemaentwurf für Inhaltsdaten und bei der Recherche zu Regelgrenzfällen wie *Calm Emotions* und Immunitäten. Die Vorschläge wurden nicht unverändert übernommen:

1. Bei einer frühen Zauberimplementierung wurde gemeinsam mit der KI ein Fahrplan erstellt. Der Code wurde jedoch schnell unübersichtlich. Eine Empfehlung, zunächst weitere repräsentative Zauber zu implementieren, wurde verworfen, um die problematische Struktur nicht weiter auszubauen.

2. In der experimentellen Phase existierten nur ein Spieler und ein Gegner. Daraus entstand die Annahme einer primären Kreatur und parallel entwickelte Logik für Spieler, Verbündete und Gegner. Diese historisch gewachsene Struktur musste später vereinheitlicht werden.
3. Bei Zuständen verhinderte Domänenwissen eine falsche Abstraktion: *Unconscious* impliziert *Incapacitated* und verursacht *Prone*. Endet *Unconscious*, endet das davon abhängige *Incapacitated*; *Prone* kann dagegen unabhängig bestehen bleiben.
4. Regelprosa wird nicht zur Laufzeit geparkt. Stattdessen werden ausführbare Mechaniken in JSON angereichert. Dadurch bleibt weniger Parsing-Logik im Programmcode verborgen.
5. Bei generischen Kreaturentyp-Anforderungen und der Modellierung von Immunität gegenüber der Unterdrückung eines Zustands wurden erste KI-Vorschläge anhand des Regeltexts überprüft und korrigiert.

3.6 Code-Generierung

Vorgabe: Erläutern, welche Bestandteile wie programmiert wurden.

Für die Entwicklung wurde OpenAI Codex eingesetzt.

3.6.1 KI-generierte Bestandteile

Vorgabe: Aufführen, welche Module, Funktionen oder Tests maßgeblich mit KI-Unterstützung erstellt wurden, und begründen, weshalb sich der KI-Einsatz dafür eignete.

- Übertragung verbleibender Regelprosa in strukturierte JSON-Daten für Zauber
- schrittweise Implementierung von Regeln wie Trefferpunkten, Angriffen und Zaubern

TODO: Konkrete Module, Funktionen und Tests mit Commit- oder Dateiverweisen ergänzen.

3.6.2 Selbst programmierte Bestandteile

Vorgabe: Kernlogiken, komplexe Workarounds oder projektspezifische Funktionen beschreiben, die vollständig selbst programmiert wurden, und die Notwendigkeit der eigenen Programmierleistung begründen.

- eigenständige Bewertung und Korrektur der Repository-Struktur
- fachliche Prüfung der von der KI vorgeschlagenen Regelmodelle

TODO: Konkrete selbst programmierte Bestandteile ergänzen.

3.7 Prompt-Engineering und Iteration

Vorgabe: Die Zusammenarbeit mit der KI beschreiben. Insbesondere erläutern, ob generierter Code direkt übernommen oder in kritischen Iterationsschleifen geprüft und überarbeitet wurde. Dazu gehören manuelle Korrekturen von Halluzinationen und veralteter Syntax.

TODO: Ein bis zwei konkrete Prompt- und Überarbeitungszyklen dokumentieren.

4 Ergebnisse

4.1 Funktionsnachweis

TODO: Testergebnisse, unterstützte Mechaniken und reproduzierbare Qualitätsprüfungen dokumentieren.

4.2 Beispieldurchlauf

TODO: Ein Szenario auswählen und den Ablauf anhand von Screenshots oder Logauszügen darstellen.

5 Diskussion und Fazit

5.1 Zielerreichung

Vorgabe: Bewerten, inwiefern das gesetzte Ziel erreicht wurde.

TODO: Erfolgskriterien aus Abschnitt 1.2 einzeln bewerten.

5.2 Herausforderungen und kritische Reflexion

TODO: Technische und organisatorische Herausforderungen zusammenfassen.

5.3 Abweichungen vom ursprünglichen Implementierungsplan

TODO: Abweichungen, Ursachen und Auswirkungen dokumentieren.

5.4 Kritische Würdigung

Vorgabe: Reflektieren, was gut lief, welche unerwarteten technischen Probleme oder Schwierigkeiten bei der Zusammenarbeit mit der KI auftraten und was bei einem zukünftigen Projekt anders gemacht würde.

TODO: Stärken, Schwächen und konkrete Verbesserungen gegenüberstellen.

5.5 Ausblick

Vorgabe: Der Ausblick darf auch eine subjektive Einschätzung enthalten.

TODO: Nächste technische Schritte und mögliche Weiterentwicklung erläutern.

6 Literatur- und Quellenverzeichnis

Vorgabe: Dieser Abschnitt ist nur erforderlich, wenn zitierfähige Quellen verwendet wurden.

- **TODO:** System Reference Document 5.2.1 bibliografisch vollständig angeben.
- **TODO:** Dokumentationen und weitere zitierte Quellen ergänzen.

7 Anhang

Vorgabe: In den Anhang gehören Inhalte, die für den Haupttext zu umfangreich sind.

7.1 Bedienungsanleitung

TODO: Installation, Programmstart, Szenarioauswahl und Bedienung dokumentieren.

7.2 Ausgewählte Code-Snippets

Vorgabe: Die ausgewählten Code-Snippets sollten jeweils nicht länger als eine Seite sein.

TODO: Repräsentative Codeausschnitte auswählen und erläutern.